

Software Metrics & Risk Handling

for

MiniOS Design

Prepared by Group - 2

Group 2 Members :
Aditya Sharma (B24CS1119)
Mehta Taksh Ashish (B24CS1115)
Ved Mitra Verma (B24CS1086)
Mayank Tiwari (B24CS1098)

03-04-2026

Contents

1	Section - I: Results & Analyses	2
1.1	Intermediate COCOMO	2
1.2	Halstead Metric	2
1.3	Function Point Analysis	3
1.4	Cumulative Flow Diagram	4
1.5	Throughput Report	4
1.6	Sprint Burndown Chart	5
1.7	Sprint Burnup Chart	5
1.8	Metrics	6
1.8.1	ABC Metric (Assignment-Branch-Condition)	6
1.8.2	Cyclomatic Complexity (V(G))	6
2	Section II: Risk Assessment and Analysis	6
2.1	Probability Estimation Model	6
2.1.1	Justification of the Model	7
2.1.2	Probability Mapping	7
2.2	Risk-wise Justification	7
2.2.1	R1: Copy-On-Write (CoW) Race Condition	7
2.2.2	R2: Memory Inconsistency Across Subsystems	8
2.2.3	R3: Memory Fragmentation	8
2.2.4	R4: Deadlocks Due to Synchronization	9
2.2.5	R5: Scheduler Instability (α, β Parameters)	9
2.2.6	R6: Garbage Collection Timing Conflict	9
2.2.7	R7: QEMU Memory Constraints	10
2.3	Risk Assessment Table	10
2.3.1	Justification of Probability Mapping	10
2.3.1.1	Example Conversions	10
2.3.2	Impact Mapping	11
2.3.2.1	Example Conversions	11
2.4	Risk Assessment Matrix	11
2.4.1	Risk Level Calculation	11
2.4.2	Risk Assessment Matrix Placement	12
2.4.3	Risk Response Strategy	13
3	Section III: Mapping Risk Assessment Matrix to Sprint Plan	13
3.1	Mapping Logic	13
3.2	Risk $\rightarrow$ Sprint Mapping Table	13
3.3	Detailed Integration with Sprint Plan	13
3.3.1	Sprint 1: Foundation & Environment Setup	13
3.3.2	Sprint 2: Core Logic Implementation	14
3.3.3	Sprint 3: Integration & Synchronization (Critical Phase)	14
3.3.4	Sprint 4: Testing, Polish & Demo	15
4	References	15

1 Section - I: Results & Analyses

Codebase Size Configuration:

- Extracted Added/Modified Lines from xv6 Base: 41,424
- Applied Reuse Factor: 5% (Given xv6 is a heavily reused existing codebase)
- **Effective Lines of Code (LOC): 2,071 (KLOC = 2.071)**

Note: This adaptation factor correctly aligns the metrics to represent a realistic effort constraint for a 4-person team over a 4-month semester limits.

1.1 Intermediate COCOMO

The Constructive Cost Model (COCOMO) estimates the required effort and schedule. The “Organic” mode was selected, mapping to small projects with small, familiar teams.

Factors Used:

- Scale Factors: $a_b = 3.2$, $b_b = 1.05$, $c_b = 2.5$, $d_b = 0.38$
- Effort Adjustment Factor (EAF): 1.2035 (Derived from Reliability=1.15, Complexity=1.15, Tool Use=0.91)

Calculated Results:

- **Effort:** 8.27 Person-Months
- **Development Time:** 5.58 Months
- **Team Size Required:** 1.48 Developers

1.2 Halstead Metric

Halstead measures complexity mathematically based on discrete operators and operands distributed logically recursively across core C files.

Formulas Used:

- Vocabulary (n) = $n_1 + n_2$
- Length (N) = $N_1 + N_2$
- Volume (V) = $N \times \log_2(n)$
- Difficulty (D) = $(n_1/2) \times (N_2/n_2)$
- Effort (E) = $D \times V$

Extracted Parameters:

- Unique Operators (n_1): 13 (*e.g.*, -, for, else, while, return, if, i, =, *, !, ., /, +)
- Total Operators (N_1): 3263

- Unique Operands (n_2): 851
- Total Operands (N_2): 7319

Calculated Results:

- **Vocabulary** (n): 864
- **Length** (N): 10582
- **Volume** (V): 103,226.22
- **Difficulty** (D): 55.90
- **Effort** (E): 5,770,661.05

1.3 Function Point Analysis

Function Point Analysis calculates functional software size derived from logic representations.

Factors & Formulas Used:

- Weight Adjustments: Internal Logical File (ILF) Weight = 7, External Input (EI) Weight = 4
- Value Adjustment Factor (VAF) = $0.65 + (0.01 \times 14 \times 3) = 1.07$

Extracted Parameters:

- Internal Logical Files (Structs identified): 113
- External Inputs/Outputs (Syscalls identified): 143

Calculated Results:

- **Unadjusted Function Points (UFP)**: 1363
- **Function Points (FPA)**: 1458.41

1.4 Cumulative Flow Diagram

The Cumulative Flow Diagram (CFD) visualizes task aggregation throughout the project lifecycle against the baseline capacity requirements.

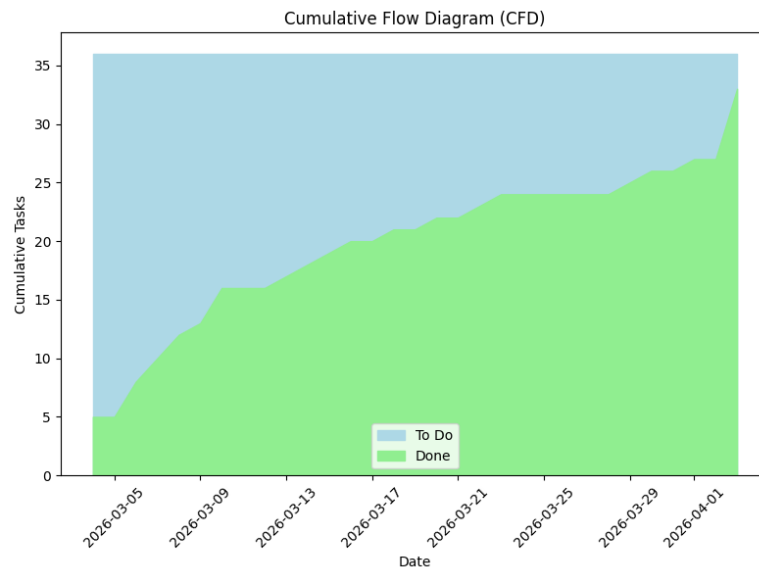


Figure 1: Cumulative Flow Diagram

1.5 Throughput Report

The Throughput report identifies timeline velocity mapping precisely when issues were migrated successfully from 'In-Progress' to 'Done'.

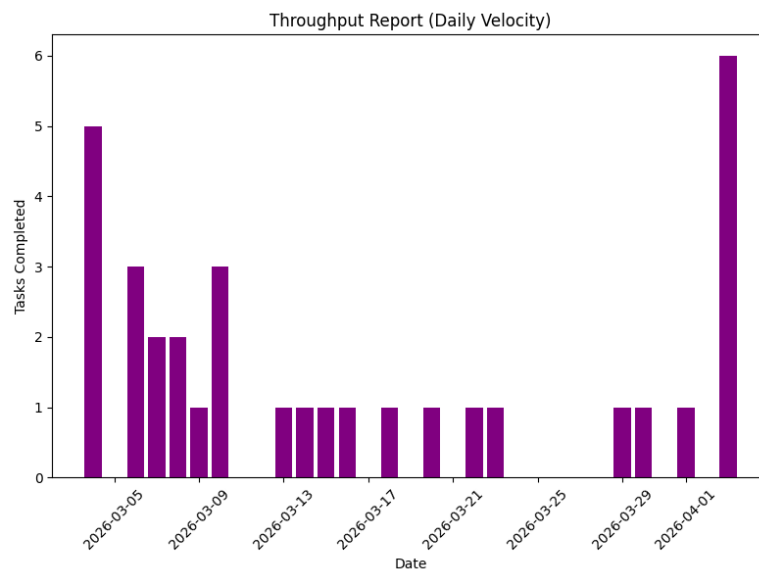


Figure 2: Throughput Report (Daily Velocity)

1.6 Sprint Burndown Chart

The Sprint Burndown captures daily sprint variance of open items falling recursively toward an ideal targeted schedule.

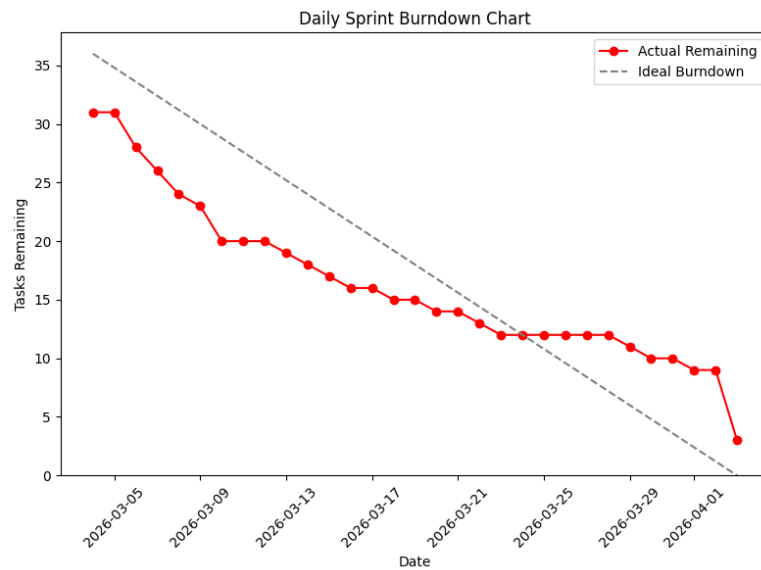


Figure 3: Daily Sprint Burndown Chart

1.7 Sprint Burnup Chart

Our Burnup highlights accumulative achievements ascending continuously toward total scope delivery.

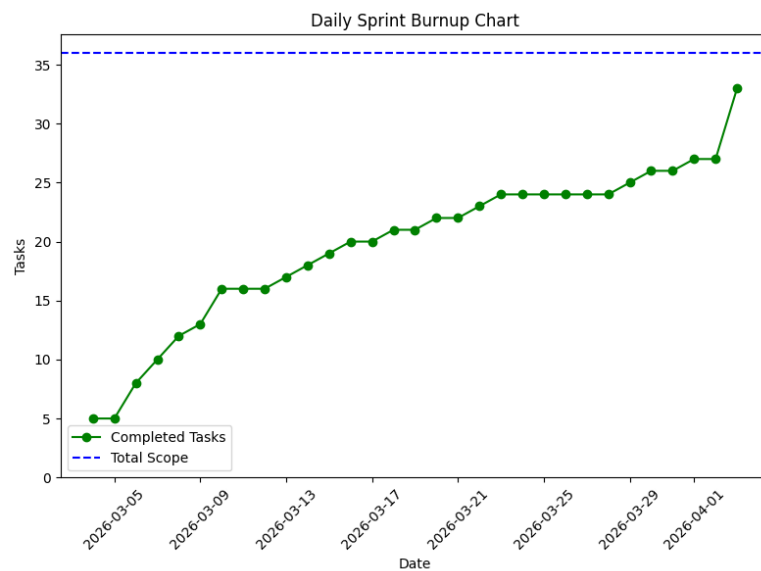


Figure 4: Daily Sprint Burnup Chart

1.8 Metrics

1.8.1 ABC Metric (Assignment-Branch-Condition)

The ABC framework evaluates complexity through dimensions representing internal logical flow boundaries.

- **Formula Used:** $ABC\ Magnitude = \sqrt{A^2 + B^2 + C^2}$
- Assignments (A): 458
- Branches (B): 256
- Conditions (C): 705
- **ABC Magnitude:** 878.82

1.8.2 Cyclomatic Complexity ($V(G)$)

Cyclomatic complexity offers a quantitative tracking path mechanism for identifying non-linear control logic structures routing.

- **Formula Used:** $V(G) = \text{Edges} - \text{Nodes} + 2$ (Simplified functionally as: Branches + Conditions + 1 per logic flow)
- **Total Nodes (N):** 1544
- **Total Edges (E):** 1896
- **Total $V(G)$ Index metric:** 372

2 Section II: Risk Assessment and Analysis

2.1 Probability Estimation Model

The probability of each identified risk is computed using a structured multi-factor model:

$$P = \frac{F + C + E}{3}$$

Where:

- F (Frequency): Likelihood of the risk condition occurring during system execution
- C (Complexity): Implementation and debugging difficulty associated with the risk
- E (Exposure): Extent of system components affected when the risk occurs

2.1.1 Justification of the Model

The adopted model is grounded in established software engineering and risk management principles:

- Barry W. Boehm, “Software Risk Management: Principles and Practices,” IEEE Software, 1991.
- IEEE Standard 1540-2001, “IEEE Standard for Software Life Cycle Processes – Risk Management.”
- Roger S. Pressman and Bruce R. Maxim, “Software Engineering: A Practitioner’s Approach,” Eighth Edition, McGraw-Hill Education, 2014.

Based on these principles:

- Frequency represents operational likelihood
- Complexity captures technical uncertainty
- Exposure represents the extent of impact

Thus, the model provides a systematic and academically grounded method for estimating probability rather than relying on arbitrary assumptions.

2.1.2 Probability Mapping

Average Score	Probability Level
1.0 – 1.5	Low (0.3)
1.6 – 2.3	Medium (0.5–0.6)
2.4 – 3.0	High (0.7–0.9)

2.2 Risk-wise Justification

2.2.1 R1: Copy-On-Write (CoW) Race Condition

Why this is a Risk

The SRS specifies that memory pages are shared after a fork operation and managed using reference counting mechanisms. This introduces shared mutable state between processes.

Concepts from Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, Tenth Edition, John Wiley & Sons, 2018 and Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 2018 explain that concurrent access to shared memory without proper synchronization leads to race conditions.

Thus, the combination of shared pages and concurrent writes makes this a fundamental risk.

F, C, E Justification

- F = 3 (High): Fork and write operations are common execution paths in operating systems.

- C = 3 (High): Requires precise synchronization mechanisms such as locks or atomic operations.
- E = 2 (Medium): Limited to shared pages but can propagate inconsistencies.

$$P = \frac{3 + 3 + 2}{3} = 2.67 \rightarrow \text{High (0.7)}$$

2.2.2 R2: Memory Inconsistency Across Subsystems

Why this is a Risk

The SRS describes multiple interacting subsystems including scheduler, memory manager, garbage collector, and file system.

Frederick P. Brooks Jr., *The Mythical Man-Month*, and Ian Sommerville, *Software Engineering*, highlight that integration of multiple components is a major source of system failures due to interaction complexity.

Therefore, maintaining consistency across subsystems is a significant risk.

F, C, E Justification

- F = 2 (Medium): Occurs mainly during integration.
- C = 3 (High): Requires maintaining consistent state across modules.
- E = 2 (Medium): Affects shared resources.

$$P = \frac{2 + 3 + 2}{3} = 2.33 \rightarrow \text{Medium-High (0.6)}$$

2.2.3 R3: Memory Fragmentation

Why this is a Risk

The SRS specifies dynamic memory allocation and compaction requirements.

Donald Knuth and OS literature describe fragmentation as an unavoidable consequence of dynamic allocation.

F, C, E Justification

- F = 3 (High): Occurs continuously.
- C = 2 (Medium): Known mitigation techniques exist.
- E = 2 (Medium): Affects performance.

$$P = \frac{3 + 2 + 2}{3} = 2.33 \rightarrow \text{Medium-High (0.6)}$$

2.2.4 R4: Deadlocks Due to Synchronization

Why this is a Risk

The system involves multiple synchronized components.

Operating system theory explains that deadlocks arise when multiple locks and shared resources interact under certain conditions.

F, C, E Justification

- F = 2 (Medium): Occurs under specific conditions.
- C = 3 (High): Hard to detect and debug.
- E = 2 (Medium): Can halt execution.

$$P = \frac{2 + 3 + 2}{3} = 2.33 \rightarrow \text{Medium-High (0.6)}$$

2.2.5 R5: Scheduler Instability (α, β Parameters)

Why this is a Risk

The SRS defines a dynamic priority scheduler based on parameters α and β .

Improper tuning can lead to starvation or inefficiency.

F, C, E Justification

- F = 2 (Medium): Occurs during tuning.
- C = 2 (Medium): Moderate complexity.
- E = 2 (Medium): Affects scheduling efficiency.

$$P = \frac{2 + 2 + 2}{3} = 2 \rightarrow \text{Medium (0.5)}$$

2.2.6 R6: Garbage Collection Timing Conflict

Why this is a Risk

Garbage collection executes during idle CPU cycles, requiring coordination with the scheduler.

Improper scheduling can interfere with execution.

F, C, E Justification

- F = 2 (Medium): Occurs during GC cycles.
- C = 2 (Medium): Requires coordination.
- E = 2 (Medium): Affects performance.

$$P = \frac{2 + 2 + 2}{3} = 2 \rightarrow \text{Medium (0.5)}$$

2.2.7 R7: QEMU Memory Constraints

Why this is a Risk

The system runs in a QEMU environment with fixed resource limits.
These constraints are predictable but restrictive.

F, C, E Justification

- F = 1 (Low): Occurs only when limits are exceeded.
- C = 1 (Low): Easy to manage.
- E = 2 (Medium): Can affect execution.

$$P = \frac{1 + 1 + 2}{3} = 1.33 \rightarrow \text{Low (0.3)}$$

2.3 Risk Assessment Table

#	Risks	Probability (%)	Impact	Level
1	Copy-On-Write Race Condition	75	C	High
2	Memory Inconsistency Across Subsystems	60	S	Medium
3	Memory Fragmentation	65	Mo	Medium
4	Deadlocks due to Synchronization	60	C	High
5	Scheduler Instability (α , β tuning)	50	Mo	Medium
6	Garbage Collection Timing Conflict	50	Mo	Medium
7	QEMU Memory Constraints	25	Mi	Low

2.3.1 Justification of Probability Mapping

Probability Range	Interpretation
0–20	Very Unlikely
21–40	Unlikely
41–60	Even Likelihood
61–85	Likely
86–100	Very Likely

2.3.1.1 Example Conversions

- **CoW Race Condition** $\rightarrow$ 75%
 $\rightarrow$ falls in 61–85 $\rightarrow$ Likely
- **Memory Inconsistency** $\rightarrow$ 60%
 $\rightarrow$ boundary of Even likelihood
- **Fragmentation** $\rightarrow$ 65%
 $\rightarrow$ Likely (continuous occurrence)

- **Deadlocks** → 60%
→ Even likelihood (conditional but serious)
- **QEMU Constraint** → 25%
→ Unlikely

These percentage values are derived from the probability model:

$$P = \frac{F + C + E}{3}$$

and then scaled to the defined percentage bands.

2.3.2 Impact Mapping

Code	Meaning
C	Critical
S	Serious
Mo	Moderate
Mi	Minor

2.3.2.1 Example Conversions

- **CoW Race Condition** → 75%
→ falls in 61–85 → Likely
- **Memory Inconsistency** → 60%
→ boundary of Even likelihood
- **Fragmentation** → 65%
→ Likely (continuous occurrence)
- **Deadlocks** → 60%
→ Even likelihood (conditional but serious)
- **QEMU Constraint** → 25%
→ Unlikely

These percentage values are derived from the probability model:

$$P = \frac{F + C + E}{3}$$

and then scaled to the defined percentage bands.

2.4 Risk Assessment Matrix

2.4.1 Risk Level Calculation

$$\text{Risk Level} = \text{Impact} \times \text{Probability}$$

Combination	Level
High Impact + High Probability	High
Medium Impact + Medium Probability	Medium
Low Impact + Low Probability	Low

2.4.2 Risk Assessment Matrix Placement

Risk	Likelihood	Impact	Zone
CoW Race Condition	Likely (4)	Critical (D/E)	Extreme / High
Memory Inconsistency	Occasional (3)	Serious (C/D)	Medium
Fragmentation	Likely (4)	Moderate (C)	Medium
Deadlocks	Occasional (3)	Critical (D)	High
Scheduler Instability	Occasional (3)	Moderate (C)	Medium
GC Conflict	Occasional (3)	Moderate (C)	Medium
QEMU Constraint	Unlikely (2)	Minor (B)	Low

Risk Assessment Matrix (MiniOS - All Risks Included)

	A Insignificant	B Marginal	C Moderate	D Critical	E Catastrophic
5 Definite					
4 Likely			Memory Fragmentation	CoW Race Condition	
3 Occasional			Scheduler Instability GC Conflict	Deadlocks Memory Inconsistency	
2 Seldom		QEMU Constraint			
1 Unlikely					

Figure 5: Risk Assessment Matrix

2.4.3 Risk Response Strategy

Risk	Strategy
CoW Race Condition	TREAT (REDUCE) – use synchronization (spinlocks)
Memory Inconsistency	TREAT (REDUCE) – define shared interfaces
Fragmentation	TREAT (REDUCE) – compaction
Deadlocks	TREAT (REDUCE) – lock ordering
Scheduler Instability	TOLERATE + TREAT – tuning
GC Conflict	TREAT (REDUCE) – idle scheduling
QEMU Constraint	TOLERATE (ACCEPT)

3 Section III: Mapping Risk Assessment Matrix to Sprint Plan

3.1 Mapping Logic

The risk assessment matrix categorizes risks based on likelihood and impact, which determines:

- Priority of mitigation
- When mitigation should occur in the development lifecycle

Risk Level	Sprint Strategy
High	Address early + continuously validate
Medium	Address during development + integration
Low	Monitor and handle if triggered

3.2 Risk → Sprint Mapping Table

Risk	Level	Mitigation Phase	Sprint Mapping
CoW Race Condition	High	Early + Integration	Sprint 2, 3
Deadlocks	High	Integration + Testing	Sprint 3, 4
Memory Inconsistency	Medium	Integration	Sprint 3
Memory Fragmentation	Medium	Design + Integration	Sprint 1, 3
Scheduler Instability	Medium	Development + Testing	Sprint 2, 4
GC Timing Conflict	Medium	Integration	Sprint 3
QEMU Constraint	Low	Setup Phase	Sprint 1

3.3 Detailed Integration with Sprint Plan

3.3.1 Sprint 1: Foundation & Environment Setup

Risks Addressed

- QEMU Constraints
- Memory Fragmentation (design stage)

How Mitigation Happens

- Environment setup defines memory limits → reduces QEMU risk
- Initial memory structures designed → prevents fragmentation issues later

Preventive risk handling phase

3.3.2 Sprint 2: Core Logic Implementation

Risks Addressed

- CoW Race Condition (initial handling)
- Scheduler Instability

How Mitigation Happens

- CoW implemented with reference counting → base for safe memory sharing
- Scheduler logic implemented → parameter tuning begins

Risks are introduced but controlled

3.3.3 Sprint 3: Integration & Synchronization (Critical Phase)

Risks Addressed

- CoW Race Condition (fully handled)
- Deadlocks
- Memory Inconsistency
- GC Timing Conflict
- Memory Fragmentation (runtime handling)

How Mitigation Happens

From SRS:

- Shared interfaces defined
- GC scheduled during idle cycles
- Memory compaction integrated
- Cross-feature conflicts resolved

This sprint directly corresponds to highest risk zone in matrix

3.3.4 Sprint 4: Testing, Polish & Demo

Risks Addressed

- Deadlocks (final validation)
- Scheduler Instability (performance validation)
- Overall system stability

How Mitigation Happens

- Stress testing ensures:
 - No kernel panic
 - No starvation
 - Stable memory behavior

Risk validation and closure phase

4 References

[Progress Tracker Sheet](#)

[Metrics Folder \(includes code, results and scripts\)](#)

[SWE Project Super Folder](#)

[Github Repository](#)